

PYTHON MASTER STUDY BOOK

A Complete Textbook for Programming with Python (Course Code
2501IT42)

VOLUME 4

UNIT IV — Object-Oriented Programming and File Operations

This volume covers classes, objects, inheritance and polymorphism with real-world analogies and diagrams, followed by a complete, practical treatment of reading and writing files.

UNIT IV

Object-Oriented Programming and File Operations

Learning Objectives (CO4)

- Understand classes and objects, and why OOP is used to model real-world things in code.
- Write constructors, instance/class attributes, and methods.
- Use inheritance to reuse and extend existing classes, and understand method overriding.
- Understand polymorphism and operator overloading.
- Read from and write to files safely, using the correct mode and the with statement.
- Use seek() and tell() to understand and control the file pointer.

Prerequisites

Units I-III — especially functions (methods are just functions that belong to a class) and the idea of grouping related data and logic together.

Part A — Object-Oriented Programming

1. Class, Object, and Instance

A **class** is a blueprint (or template) that describes what data and behaviour something should have. An **object** is one actual thing built from that blueprint. Every object built from a class is also called an **instance** of that class.

Real-world analogy: a class is like the architectural blueprint for a house. A blueprint isn't a house you can live in — it just describes what every house built from it will look like. Each actual house built from that blueprint is an object (instance). You can build many houses (objects) from one blueprint (class), and each house can be painted a different colour (each object can hold its own data).

```
class Dog:                                # CLASS -- the blueprint
    pass

d1 = Dog()                                # d1 is an OBJECT / INSTANCE of Dog
```

```
# d2 is a DIFFERENT object, also an instance of Dog
d2 = Dog()
print(type(d1))           # <class '__main__.Dog'>
# False -- two separate objects, even though same class
print(d1 is d2)
```

2. The Constructor: `__init__()`

A **constructor** is a special method that runs AUTOMATICALLY every time a new object is created, used to set up that object's starting data. In Python the constructor is always named `__init__`. The first parameter of every method, by convention named `self`, refers to the specific object being worked on.

```
class Student:
    # constructor -- runs on Student(...)
    def __init__(self, name, marks):
        # INSTANCE ATTRIBUTE -- unique to this object
        self.name = name
        self.marks = marks

# __init__ runs automatically: self=s1, name="Asha", marks=91
s1 = Student("Asha", 91)
# a separate call: self=s2, name="Ravi", marks=78
s2 = Student("Ravi", 78)

print(s1.name, s1.marks)      # Asha 91
print(s2.name, s2.marks)     # Ravi 78
```

COMPETITIVE EXAM / GATE FOCUS

`self` is not a Python keyword -- it is simply a convention (you could technically name it anything, but never do). When you call `Student("Asha", 91)`, Python automatically passes the new, still-empty object as the first argument to `__init__`, which is why you write `def __init__(self, name, marks)` but CALL it as `Student("Asha", 91)` -- `self` is filled in automatically and never passed explicitly by the caller.

3. Attributes and Methods

Term	Meaning
Attribute	A piece of data belonging to an object (or class), accessed as object.attribute_name.
Method	A function defined inside a class, describing something the object can DO. Called as object.method_name(...).
Instance attribute	Belongs to ONE specific object; usually set in __init__ using self.name = ...
Class attribute	Shared by ALL objects of the class; defined directly inside the class body, outside any method.

```

class Student:
    # CLASS ATTRIBUTE -- shared by every Student
    school_name = "Aditya University"

    def __init__(self, name, marks):
        # INSTANCE ATTRIBUTE -- unique per object
        self.name = name
        self.marks = marks

    # METHOD -- something a Student object can do
    def show(self):
        print(f"{self.name} ({self.school_name}): {self.marks} marks")

s1 = Student("Asha", 91)
s2 = Student("Ravi", 78)
s1.show()      # Asha (Aditya University): 91 marks
s2.show()      # Ravi (Aditya University): 78 marks

```

Class attribute vs instance attribute -- what happens if we change each

Step	Code executed	Variable state after
1	Student.school_name = 'NIT'	Changes for EVERY Student object (s1 and s2 both now show 'NIT')
2	s1.name = 'Anjali'	Changes ONLY s1; s2.name is still 'Ravi' -- instance attributes are private to their object

COMMON MISTAKES

- Forgetting `self` as the first parameter of a method -- Python will pass the object automatically regardless of what you name it, but forgetting it entirely causes a `TypeError` about the wrong number of arguments.
- Confusing class attributes with instance attributes: assigning `self.school_name = "X"` inside a method creates a NEW instance attribute on that one object, shadowing the class attribute, rather than changing it for everyone.

4. Destructor: `__del__()`

A **destructor** runs automatically when an object is about to be destroyed (typically when it is garbage collected because nothing refers to it anymore). In Python this is rarely needed explicitly, because Python manages memory automatically, but it can be used for cleanup like closing a file or a network connection.

```
class Connection:
    def __init__(self, name):
        self.name = name
        print(f"Connection to {self.name} opened")

    def __del__(self):
        print(f"Connection to {self.name} closed")

c = Connection("server1")    # prints: Connection to server1 opened
# prints: Connection to server1 closed
del c
```

COMPETITIVE EXAM / GATE FOCUS

Exactly WHEN `__del__` runs is not always predictable (it depends on Python's garbage collector), so real programs should not rely on it for critical cleanup like closing files -- use the `with` statement instead (covered later in this chapter), which guarantees cleanup at a precise point.

5. Encapsulation

Encapsulation means bundling data (attributes) and the methods that operate on that data together inside one class, and controlling how that data can be accessed from outside. The goal is to protect an object's internal state from being changed in invalid ways.

Convention	Meaning
name	Public -- accessible from anywhere, no restriction (Python's default).
<code>_name</code>	Protected (by convention only) -- a signal to other programmers 'internal use, please don't touch directly', but Python does not actually enforce this.
<code>__name</code>	Private-ish -- Python performs 'name mangling', renaming it internally to <code>_ClassName__name</code> , which makes accidental external access harder (but not impossible).

```
class BankAccount:
    def __init__(self, balance):
        # "private" -- discourages direct outside access
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    # controlled access via a method
    def get_balance(self):
        return self.__balance

acc = BankAccount(1000)
acc.deposit(500)
print(acc.get_balance())          # 1500
# AttributeError -- name has been mangled
# print(acc.__balance)
```

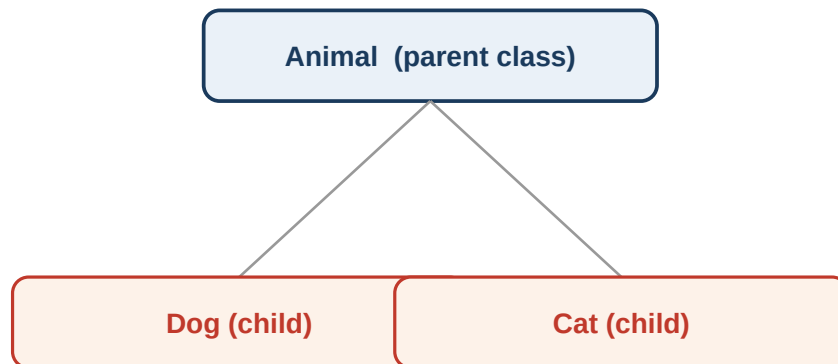
COMPETITIVE EXAM / GATE FOCUS

Python has no TRUE private attributes like Java or C++ -- double-underscore names are only 'name-mangled' (renamed to `_ClassName__attr`), and can technically still be accessed if you know the mangled name. Python's philosophy is often summarised as 'we are all consenting adults here': privacy is a strong convention, enforced by discipline rather than the language itself.

6. Inheritance

Inheritance lets a new class (the **child** or **subclass**) reuse the attributes and methods of an existing class (the **parent** or **superclass**), and add or change behaviour of its own. This avoids rewriting the same code for closely related things.

Basic (single) inheritance



Both Dog and Cat automatically get everything Animal defines, and can add their own extras.

```
class Animal:                                # PARENT / superclass
    def __init__(self, name):
        self.name = name

    def eat(self):
        print(f"{self.name} is eating")

# CHILD / subclass -- inherits from Animal
class Dog(Animal):
    # a NEW method, only Dog has this
    def bark(self):
        print(f"{self.name} says Woof!")

d = Dog("Rex")
# Rex is eating    -- inherited from Animal, Dog never defined this itself
d.eat()
d.bark()          # Rex says Woof!    -- defined directly in Dog
```

Types of Inheritance

Type	Description
Single	One child inherits from one parent (the Dog/Animal example above).

Type	Description
Multiple	One child inherits from more than one parent class at once: class C(A, B):
Multilevel	A chain: class C inherits from B, which inherits from A (grandparent -> parent -> child).
Hierarchical	Multiple children inherit from the SAME single parent (Dog and Cat both from Animal).
Hybrid	Any combination of the above types used together.

```
# Multiple inheritance
```

```
class Flyer:
    def fly(self):
        print("Flying")
```

```
class Swimmer:
    def swim(self):
        print("Swimming")
```

```
class Duck(Flyer, Swimmer):    # inherits from BOTH parents
    pass
```

```
d = Duck()
d.fly()      # Flying
d.swim()     # Swimming
```

Using super() to call the parent's version

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        # calls Animal's __init__ to set up self.name
        super().__init__(name)
        # then adds Dog's own extra attribute
        self.breed = breed
```

```
d = Dog("Rex", "Labrador")
print(d.name, d.breed)    # Rex Labrador
```

COMPETITIVE EXAM / GATE FOCUS

`super()` lets a child class REUSE the parent's `__init__` (or any other method) instead of copy-pasting its logic. This is the standard, recommended way to extend a parent constructor with extra attributes, and is a very common interview and exam pattern.

7. Method Overriding and Polymorphism

Method overriding happens when a child class defines a method with the SAME NAME as one in its parent, replacing the parent's version specifically for objects of the child class.

```
class Animal:
    def sound(self):
        print("Some generic animal sound")

class Dog(Animal):
    def sound(self):                # OVERRIDES Animal.sound()
        print("Woof!")

class Cat(Animal):
    # OVERRIDES Animal.sound() differently
    def sound(self):
        print("Meow!")

a = Animal(); a.sound()           # Some generic animal sound
d = Dog();    d.sound()           # Woof!
c = Cat();    c.sound()           # Meow!
```

Polymorphism (Greek for 'many forms') means the SAME method call can behave differently depending on the actual object it is called on. The example above already demonstrates this: calling `.sound()` produces different output depending on whether the object is an `Animal`, `Dog`, or `Cat`.

```
animals = [Animal(), Dog(), Cat()]
for a in animals:
    # each call automatically runs the RIGHT version for that class
    a.sound()
# Some generic animal sound
```

```
# Woof!
# Meow!
```

COMPETITIVE EXAM / GATE FOCUS

This 'loop that calls the same method name but gets different behaviour per object' pattern is the textbook demonstration of polymorphism, and appears constantly in both exams and real frameworks (e.g. every shape in a list can have `.area()` called on it, even though Circle, Square, and Triangle each compute area completely differently).

Operator Overloading

Python lets a class define what an operator like `+` should do for its own objects, by implementing special "dunder" (double-underscore) methods such as `__add__`.

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    # defines what Point + Point should do
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    # defines what print(point) / str(point) shows
    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Point(1, 2)
p2 = Point(3, 4)
p3 = p1 + p2          # Python calls p1.__add__(p2) automatically
print(p3)             # (4, 6)
```

Special method	What it overloads
<code>__add__(self, other)</code>	the <code>+</code> operator
<code>__sub__(self, other)</code>	the <code>-</code> operator
<code>__mul__(self, other)</code>	the <code>*</code> operator
<code>__eq__(self, other)</code>	the <code>==</code> operator

Special method	What it overloads
<code>__str__(self)</code>	<code>print(obj)</code> and <code>str(obj)</code> -- human-readable text
<code>__len__(self)</code>	<code>len(obj)</code>

MUST REMEMBER

- Class = blueprint; object/instance = a specific thing built from that blueprint.
- `__init__` is the constructor; `self` refers to the specific object a method is operating on.
- Instance attributes (`self.x`) belong to one object; class attributes are shared by every object.
- Inheritance: `class Child(Parent):` reuses and extends Parent's attributes/methods; `super()` calls the parent's version.
- Method overriding = child redefines a parent's method by using the same name.
- Polymorphism = the same method call behaves differently depending on the object's actual class.
- Operator overloading uses dunder methods like `__add__`, `__str__`, `__eq__` to define operator behaviour for your own classes.

INTERVIEW QUESTIONS

Q: What is the difference between overriding and overloading?

A: Overriding means a child class redefines a method that already exists in its parent class, with the same name and signature, to provide different behaviour. Overloading in the classic sense (multiple methods with the same name but different parameter lists) is not directly supported in Python -- Python instead achieves similar flexibility using default arguments, `*args/**kwargs`, or by overloading OPERATORS via dunder methods like `__add__`.

Q: Why does Python not have true private attributes?

A: Python favours convention and trust over strict enforcement. A single underscore signals 'internal use only' and a double underscore triggers name mangling to discourage accidental access, but neither makes an attribute truly inaccessible from outside the class.

Part B — File Operations

1. What is a File, and Opening Files

A **file** is data stored persistently on disk (unlike a variable, which disappears when the program ends). To work with a file in Python, you first **open** it, which gives you a file object you can read from or write to, and then you must **close** it when finished, to release the operating system resource and make sure everything is actually saved to disk.

```
f = open("notes.txt", "r")      # open in READ mode
content = f.read()
f.close()                      # always close when done
```

File Modes

Mode	Meaning
'r'	Read (default). File must already exist, or FileNotFoundError is raised.
'w'	Write. Creates the file if it doesn't exist; ERASES all existing content if it does.
'a'	Append. Creates the file if it doesn't exist; adds new content to the END without erasing.
'x'	Exclusive create. Fails with an error if the file already exists.
'r+'	Read AND write, file must exist.
'rb' / 'wb'	Same as r/w, but in BINARY mode (for images, PDFs, etc. instead of text).

COMPETITIVE EXAM / GATE FOCUS

Opening a file in 'w' mode on a file that already has important content is a classic and irreversible mistake: it immediately erases everything in the file, even before you write anything new. If you only want to add new content without erasing, always use 'a' mode.

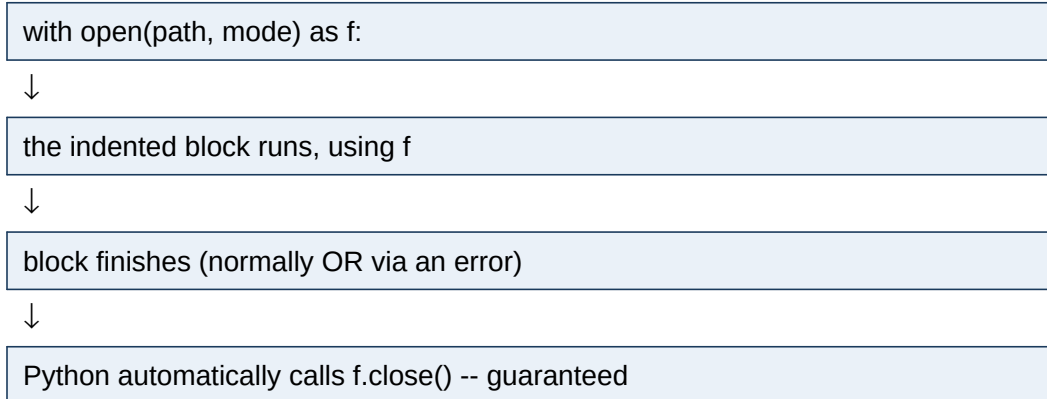
2. The with Statement (STRONGLY preferred over manual open/close)

The with statement automatically closes the file for you, even if an error happens partway through — this is far safer than remembering to call `.close()` manually.

```
with open("notes.txt", "r") as f:
    content = f.read()
# file is automatically closed here, as soon as this indented block ends -- guaranteed
```

even if an exception occurred while reading

Why 'with' is safer than manual open()/close()



COMMON MISTAKES

- Forgetting to close a file opened manually with open() -- can leave data unwritten to disk (especially with write modes) or exhaust the OS's limit on open files in long-running programs. Prefer `with` always.

3. Reading Files: read(), readline(), readlines()

Method	Returns
read()	The ENTIRE file content as one single string.
readline()	ONE line at a time (including the trailing newline character); call again for the next line.
readlines()	A LIST of every line in the file, each line as one string element.

```

with open("notes.txt", "r") as f:
    whole_text = f.read()
print(whole_text)

```

```

with open("notes.txt", "r") as f:
    first_line = f.readline()
    second_line = f.readline()

```

```

with open("notes.txt", "r") as f:
    # e.g. ['line1\n', 'line2\n', 'line3']
    lines = f.readlines()

```

MOST COMMON: loop directly over the file

```
with open("notes.txt", "r") as f:
    for line in f:
        # strip() removes the trailing \n
        print(line.strip())
```

4. Writing Files: write() and writelines()

```
with open("output.txt", "w") as f:
    # write() adds NO newline automatically -- add \n yourself
    f.write("Hello, World!\n")
    f.write("Second line\n")
```

```
lines = ["one\n", "two\n", "three\n"]
with open("output.txt", "w") as f:
    # writes each string in the list, exactly as given
    f.writelines(lines)
```

```
# APPEND -- adds to the end without erasing
with open("output.txt", "a") as f:
    f.write("Fourth line\n")
```

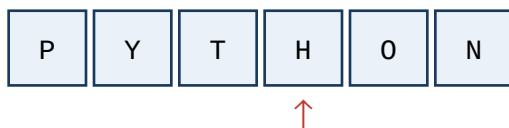
COMMON MISTAKES

- Forgetting that write() does NOT add a newline automatically -- consecutive write() calls will run together on the same line unless you include \n yourself.
- Using 'w' mode when you meant to add to an existing file -- this erases the previous content first.

5. The File Pointer: seek() and tell()

Every open file has an invisible **file pointer** (also called the file position) marking exactly where the next read or write will happen. tell() reports the pointer's CURRENT position (as a number of characters/bytes from the start). seek(offset) MOVES the pointer to a specific position.

File containing 'PYTHON' -- pointer after reading 3 characters



File pointer (position 3) shown by the arrow -- this is what tell() would return.

```
with open("data.txt", "w") as f:
    f.write("PYTHON")

with open("data.txt", "r") as f:
    # 0 -- pointer starts at the very beginning
    print(f.tell())
    chunk = f.read(3)          # reads 3 characters: "PYT"
    # 3 -- pointer has moved forward by 3
    print(f.tell())
    # move pointer back to the very start
    f.seek(0)
    # PYTHON -- reads everything again from position 0
    print(f.read())
```

COMPETITIVE EXAM / GATE FOCUS

`seek()` and `tell()` together let a program re-read parts of a file without closing and reopening it, which is especially useful for things like reading a file header, then jumping back to re-scan the body, or resuming reading from a saved bookmark position in a very large file.

6. Reading Config Files and Writing Log Files (practical use)

```
# --- config.txt ---
# host=localhost
# port=8080

config = {}
with open("config.txt", "r") as f:
    for line in f:
        line = line.strip()
        if line and not line.startswith("#") and "=" in line:
            key, value = line.split("=", 1)
            config[key.strip()] = value.strip()

print(config)      # {'host': 'localhost', 'port': '8080'}

import datetime

def log(message):
    timestamp = datetime.datetime.now()
    # APPEND -- keeps old log entries
    with open("app.log", "a") as f:
```

```
f.write(f"[{timestamp}] {message}\n")
```

```
log("Application started")  
log("User logged in")
```

Common File Errors

Error	Cause
FileNotFoundError	Trying to open a non-existent file in 'r' mode.
PermissionError	No permission to read/write the file (e.g. a read-only or locked file).
Overwritten data	Opening an existing file in 'w' mode by mistake, instead of 'a'.
Garbled/missing newlines	Forgetting \n in write() calls.

QUICK REVISION

- 'r' read (file must exist), 'w' write (erases existing content!), 'a' append (keeps existing content).
- Always prefer `with open(...)` as `f:` -- guarantees the file closes automatically, even on errors.
- `read()` -> whole file as one string. `readline()` -> one line. `readlines()` -> list of all lines.
- `write()` does NOT add `\n` automatically -- you must include it yourself.
- `tell()` reports the current pointer position; `seek(n)` moves the pointer to position `n`.

Practice Problems (Unit IV)

A. Beginner / OOP

01. Create a Time Class (syllabus: CodeChef TIMEMNG)

TRY YOURSELF

01. Create a Time Class (syllabus: CodeChef TIMEMNG)

Create a Time class that stores a duration in seconds, and can convert it to minutes and hours.

Input: seconds=3665

Output: 1 hours, 1 minutes, 5 seconds

HINT

Store seconds in `__init__`. Use integer division and modulus to split into hours/minutes/seconds.

SOLUTION

See code and explanation below.

```
class Time:
    def __init__(self, total_seconds):
        self.total_seconds = total_seconds

    def to_hms(self):
        h = self.total_seconds // 3600
        m = (self.total_seconds % 3600) // 60
        s = self.total_seconds % 60
        return h, m, s

t = Time(3665)
h, m, s = t.to_hms()
print(f"{h} hours, {m} minutes, {s} seconds")
```

This models a real-world concept (a duration) as a class with data (`total_seconds`) and behaviour (`to_hms()`) bundled together -- exactly what encapsulation means. The

arithmetic reuses the % and // operators from Unit I. $O(1)$ time.

O2. Converter Class for Unit Conversions (syllabus: CodeChef CONV01)

TRY YOURSELF

O2. Converter Class for Unit Conversions (syllabus: CodeChef CONV01)

Create a Converter class with a method to convert inches to feet (1 foot = 12 inches).

Input: inches=36

Output: 3.0 feet

HINT

One method, one formula: $\text{feet} = \text{inches} / 12$.

SOLUTION

See code and explanation below.

```
class Converter:
    def inches_to_feet(self, inches):
        return inches / 12

c = Converter()
print(c.inches_to_feet(36), "feet")
```

A Converter class is a good example of grouping several RELATED conversion methods together (you could add inches_to_cm, feet_to_meters, etc. to the same class) instead of scattering unrelated free functions around a program.

O3. Product Class with Dynamic Pricing and Stock (syllabus: CodeChef STFOOD)

TRY YOURSELF**O3. Product Class with Dynamic Pricing and Stock (syllabus: CodeChef STFOOD)**

Create a Product class that stores name, price and stock quantity, with a method to sell some quantity (reducing stock) and a method to update the price.

Input: Product('Rice', 50, 100), sell 10

Output: Remaining stock: 90

HINT

Store attributes in `__init__`. The `sell()` method should check that enough stock exists before reducing it.

SOLUTION

See code and explanation below.

```
class Product:
    def __init__(self, name, price, stock):
        self.name = name
        self.price = price
        self.stock = stock

    def sell(self, qty):
        if qty > self.stock:
            print("Not enough stock!")
            return
        self.stock -= qty
        print(f"Sold {qty} {self.name}(s). Remaining stock: {self.stock}")

    def update_price(self, new_price):
        self.price = new_price

p = Product("Rice", 50, 100)
p.sell(10)
```

This models a realistic, live-project-style object: state (name, price, stock) plus behaviour that changes that state safely (`sell()` validates before modifying stock, rather than letting

stock go negative). This validation-before-mutation pattern is standard in real OOP design.

B. Easy / File Handling

O4. Say 'Hello, World!' From a File (syllabus: HackerRank write-a-function)

TRY YOURSELF

O4. Say 'Hello, World!' From a File (syllabus: HackerRank write-a-function)

Read a line from a file and print it back exactly.

Input: greeting.txt containing: Hello, World!

Output: Hello, World!

HINT

Open the file in read mode, read one line, and print it (using `.strip()` to remove the trailing newline if needed).

SOLUTION

See code and explanation below.

```
with open("greeting.txt", "r") as f:
    line = f.readline()
print(line.strip())
```

Using ``with`` guarantees the file closes automatically. `.strip()` removes the newline character that `readline()` includes, so the printed output matches exactly.

O5. Reading From STDIN / File and Processing Lines (syllabus: HackerRank whats-your-name)

TRY YOURSELF**O5. Reading From STDIN / File and Processing Lines (syllabus: HackerRank whats-your-name)**

Read a name and print a greeting using it, either from user input or a file line.

Input: file contains: Asha

Output: Hello, Asha! How are you doing today?

HINT

Read and strip the line to remove trailing whitespace/newlines before using it in the message.

SOLUTION

See code and explanation below.

```
with open("name.txt", "r") as f:
    name = f.readline().strip()
print(f"Hello, {name}! How are you doing today?")
```

Stripping the line is essential here -- without it, the trailing newline character would appear inside the printed sentence, which is a very common beginner file-handling bug.

C. Medium / Interview-style

O6. Word Frequency Counter From a File (original problem, combines Unit II + file handling)**TRY YOURSELF****O6. Word Frequency Counter From a File (original problem, combines Unit II + file handling)**

Read a text file and print how many times each word appears, ignoring case.

Input: file contains: "the cat sat on the mat"

Output: {'the': 2, 'cat': 1, 'sat': 1, 'on': 1, 'mat': 1}

HINT

Read the whole file, split into words, lowercase each word, then use the frequency-counting pattern from Unit II.

SOLUTION

See code and explanation below.

```
freq = {}
with open("text.txt", "r") as f:
    text = f.read()
words = text.lower().split()
for w in words:
    freq[w] = freq.get(w, 0) + 1
print(freq)
```

This combines three units of learning in one small program: file reading (Unit IV), string splitting (Unit II), and the get()-based frequency pattern (Unit II) -- exactly the kind of combined-skill question that appears in university practicals and interviews.

07. Simple Class Hierarchy: Shape, Circle, Rectangle (original OOP problem)**TRY YOURSELF****07. Simple Class Hierarchy: Shape, Circle, Rectangle (original OOP problem)**

Create a Shape parent class with an area() method returning 0, and Circle and Rectangle child classes that override area() correctly.

Input: Circle(radius=3), Rectangle(w=4,h=5)

Output: Circle area: 28.26, Rectangle area: 20

HINT

Override area() in each child class using the correct formula; use math.pi for the circle.

SOLUTION

See code and explanation below.

```
import math

class Shape:
    def area(self):
        return 0

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return round(math.pi * self.radius ** 2, 2)

class Rectangle(Shape):
    def __init__(self, w, h):
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h

shapes = [Circle(3), Rectangle(4, 5)]
for s in shapes:
    print(type(s).__name__, "area:", s.area())
```

This is the classic polymorphism demonstration: the SAME loop calls `.area()` on every shape without knowing or caring which exact subclass it is, and Python automatically runs the correct overridden version for each object.

Master Comparison Tables (Unit IV)

Class vs Object

Class	Object
The blueprint / template	One actual thing built from the blueprint
Defined once with `class`	Created (instantiated) as many times as needed
class Dog:	d1 = Dog(), d2 = Dog()

Method vs Function

Function	Method
Defined independently with def, called on its own: f(x)	Defined inside a class, called on an object: obj.method(x)
No implicit first argument	Implicitly receives the object as its first argument (self)

Inheritance vs Polymorphism

Inheritance	Polymorphism
A child class reuses/extends a parent class's code	The same method call behaves differently depending on the object's actual class
About code REUSE and structure	About FLEXIBLE behaviour at the point of calling
class Dog(Animal):	for a in animals: a.sound() -- each runs its own version

read() vs readline() vs readlines()

read()	readline()	readlines()
Whole file as ONE string	ONE line per call	A LIST of every line

try/except vs try/finally (preview -- fully covered in Unit V)

try/except	try/finally
Catches and HANDLES an error if one occurs	ALWAYS runs the finally block whether or not an error occurred -- typically used for cleanup

Unit IV Mock Test

1. What runs automatically when an object is created?
(a) `__del__` (b) `__init__` (c) `__str__` (d) `__main__`
2. What does `self` refer to?
(a) the class itself (b) the specific object the method is called on (c) a keyword (d) nothing
3. Which is shared by ALL objects of a class?
(a) instance attribute (b) class attribute (c) local variable (d) parameter
4. What does `super().__init__()` do in a child class?
(a) deletes the parent (b) calls the parent's constructor (c) creates a new class (d) nothing
5. Overriding means:
(a) same method name, different behaviour in child class (b) deleting a method (c) renaming a variable (d) using `super()`
6. Which file mode ERASES existing content?
(a) 'a' (b) 'r' (c) 'w' (d) 'r+'
7. What does `f.readlines()` return?
(a) one string (b) a list of lines (c) one line (d) None
8. What does `f.tell()` return?
(a) file name (b) current pointer position (c) file size (d) file mode
9. Which correctly demonstrates polymorphism?
(a) two unrelated functions with the same name (b) a loop calling the same method name on different subclass objects (c) a single class with one method (d) importing a module
10. What does `with open(...) as f:` guarantee?
(a) faster reading (b) the file is automatically closed (c) no errors ever occur (d) the file is always in write mode

Coding Questions

- Create a `Rectangle` class with `width` and `height` attributes and a method to compute its area and perimeter.
- Create a `Vehicle` parent class and `Car/Bike` child classes that override a `describe()` method.
- Write a program to count the number of lines, words, and characters in a text file.

- Write a program that reads a file and writes only the lines containing a given keyword to a new file.
- Create a class BankAccount with deposit(), withdraw() (which should refuse to overdraw), and get_balance() methods, demonstrating encapsulation.

Answer Key

Q	Answer	Q	Answer
1	(b) __init__ -- the constructor	6	(c) 'w' -- erases existing content
2	(b) the specific object the method is called on	7	(b) a list of lines
3	(b) class attribute -- shared by all instances	8	(b) current pointer position
4	(b) calls the parent's constructor	9	(b) same call, different behaviour per subclass
5	(a) same method name, different behaviour in child class	10	(b) the file is automatically closed

End of Unit IV. Unit V (Errors, Exceptions, and Graphical User Interfaces) will follow next, with the same visual and interactive structure.